

BTVN#3 – Dựn Agent đặt vé máy bay bằng LangChain

Môn học: SE373

Họ và tên: Trần Thế Đăng

MSSV: 23520238

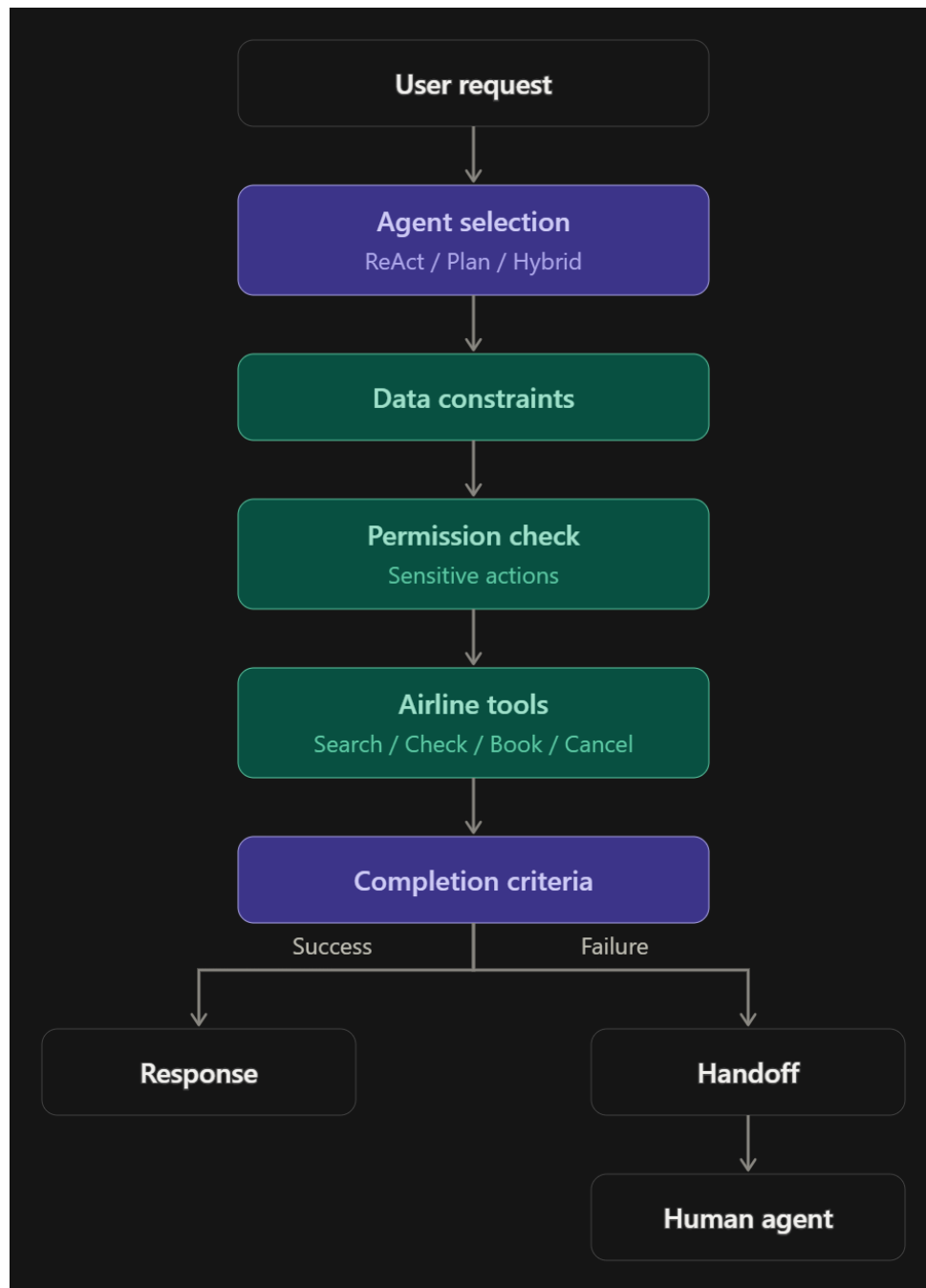
1. Mục tiêu

Mục tiêu của bài tập là xây dựng một hệ thống Agent đặt vé máy bay bằng LangChain, đồng thời áp dụng các cơ chế kiểm soát Agent thông qua Harness.

Hệ thống gồm:

- Data Constraints – kiểm tra dữ liệu đầu vào.
- Completion Criteria – kiểm tra tác vụ đã hoàn thành hay chưa.
- Permission Check – kiểm tra quyền thực hiện các hành động nhạy cảm.
- Handoff – chuyển sang human agent khi Agent không thể tiếp tục.
- Ba kiến trúc Agent:
 - ReAct
 - Plan-then-Execute
 - Hybrid

2. Kiến trúc hệ thống



Hình 1. Tổng quan kiến trúc hệ thống

Mô tả

Hệ thống nhận yêu cầu từ người dùng và sử dụng một trong ba thiết kế Agent để xử lý. Các yêu cầu được kiểm tra dữ liệu trước khi thực hiện. Với những hành động nhạy cảm như đặt hoặc hủy vé, hệ thống kiểm tra quyền và yêu cầu người dùng xác nhận. Sau khi thực hiện tool, Completion Harness kiểm tra kết quả thực tế. Nếu không thể tiếp tục, Handoff Harness sẽ chuyển tác vụ cho human agent.

3. Các Harness được cài đặt

3.1. Data Constraints

DataConstraintHarness kiểm tra:

- Mã sân bay có hợp lệ hay không.
- Sân bay đi và đến có giống nhau hay không.
- Ngày có đúng định dạng YYYY-MM-DD.
- Ngày bay không được ở quá khứ.
- Số lượng hành khách nằm trong khoảng cho phép.

Ví dụ:

```
(.venv) PS C:\Users\ADMIN\Desktop\SE373\BTVN#3 Agent Flight> python -m tests.test_harness
=====
TEST 1 – DATA CONSTRAINTS
=====
Valid: True
Errors: []

=====
TEST 2 – INVALID DATA
=====
Valid: False
- Invalid origin airport: XXX
- Passengers must be <= 9.
- Date must use YYYY-MM-DD format.
```

Hình 2. Test data constraints

Kết quả cho thấy hệ thống có thể phát hiện và trả về các lỗi dữ liệu trước khi thực hiện tác vụ.

3.2. Permission Check

PermissionHarness kiểm soát các hành động nhạy cảm:

- book_flight
- cancel_booking

Đối với đặt vé, Agent phải có xác nhận rõ ràng từ người dùng.

Ví dụ:

```
=====
TEST 3 – PERMISSION
=====
Allowed: False
Reason: Action 'book_flight' requires explicit user confirmation.
Allowed: True
Reason: User confirmation granted for 'book_flight'.
```

Hình 3. Test permission

3.3. Completion Criteria

CompletionHarness kiểm tra kết quả thực tế của tool.

Các trường hợp:

- Search: kiểm tra kết quả có chứa thông tin chuyến bay.
- Booking: kiểm tra BOOKING_SUCCESS.
- Cancellation: kiểm tra CANCELLATION_SUCCESS.

Ví dụ:

```
=====
TEST 4 – COMPLETION
=====
Completed: True
Reason: Booking completed successfully.
Completed: False
Reason: Booking was not completed.
```

Hình 4. Test completion criteria

Điều này giúp hệ thống không chỉ dựa vào câu trả lời của LLM để quyết định rằng tác vụ đã thành công.

3.4. Handoff

HandoffHarness quyết định khi nào cần chuyển tác vụ cho human agent.

Handoff xảy ra khi:

- Dữ liệu không hợp lệ.
- Không có quyền thực hiện hành động.
- Có quá nhiều lỗi.
- Tác vụ không hoàn thành.

Ví dụ:

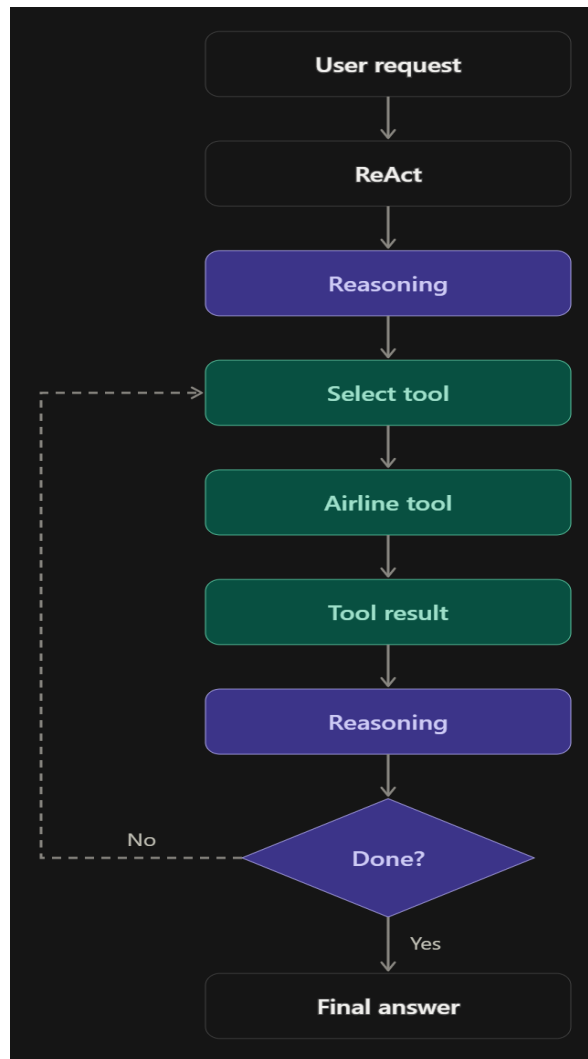
```
=====
TEST 5 – HANDOFF
=====
Handoff: False
Reason: Task completed successfully.
Handoff: True
Reason: Agent does not have permission to continue.
Target: human_agent
```

Hình 5. Test handoff

4. Các Agent được triển khai

4.1. ReAct Agent

ReAct Agent sử dụng cách tiếp cận **Reasoning + Acting**.



Hình 6. ReAct Agent

Agent được kết nối với các tool:

- search_flights
- check_flight
- book_flight
- cancel_booking

Ưu điểm

- Linh hoạt.
- Có thể phản ứng với kết quả của từng tool.
- Phù hợp với những tác vụ cần nhiều bước và thông tin chưa biết trước.

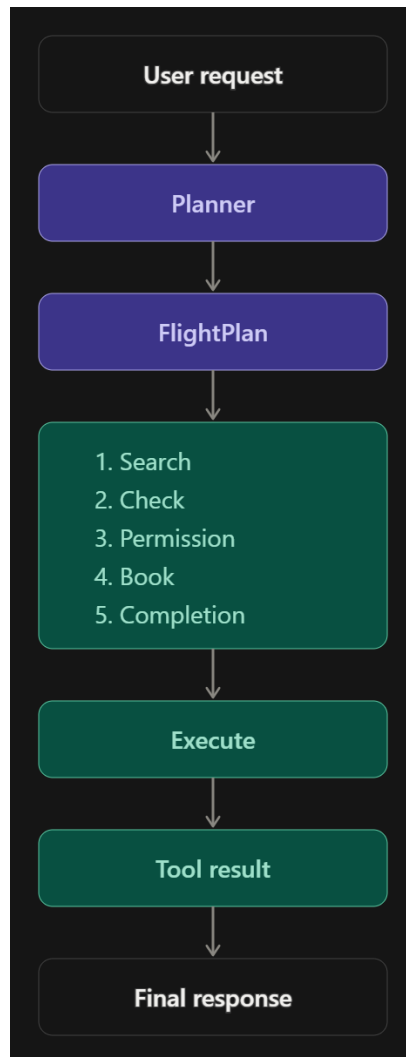
Nhược điểm

- Luồng thực hiện khó dự đoán hơn.
- Phụ thuộc nhiều vào khả năng reasoning của LLM.

- Nếu chỉ dựa vào prompt thì một số ràng buộc có thể không được đảm bảo tuyệt đối.

4.2. Plan-then-Execute Agent

Agent này chia quá trình thành hai giai đoạn: LLM trước tiên tạo một FlightPlan, sau đó chương trình thực hiện từng bước trong kế hoạch.



Hình 7. Plan-then-Execute Agent

Ví dụ:

1. Search flights
2. Check selected flight
3. Check permission
4. Book flight
5. Check completion

Ưu điểm

- Luồng xử lý rõ ràng.

- Dễ kiểm soát.
- Dễ debug.
- Dễ kiểm tra permission và completion.

Nhược điểm

- Nếu tình huống thay đổi trong quá trình thực hiện, kế hoạch ban đầu có thể không còn phù hợp.
- Có thêm bước tạo plan nên có thể tốn thêm thời gian gọi LLM.

4.3. Hybrid Agent

Hybrid Agent kết hợp hai cách tiếp cận Plan-then-Execute và ReAct. Trước tiên, LLM tạo ra một HybridPlan có cấu trúc gồm thông tin chuyển bay, mục tiêu của tác vụ và các bước cần thực hiện. Sau đó, phần thực thi sử dụng ReAct Agent để tương tác với các tool dựa trên kế hoạch đã tạo.

Quy trình xử lý của Hybrid Agent gồm các bước chính:



Hình 8. Hybrid Agent

Một điểm quan trọng trong thiết kế này là việc xác nhận đặt vé không được suy luận hoàn toàn bởi LLM. Chương trình sử dụng logic kiểm tra trực tiếp nội dung người dùng để xác định họ có thực sự xác nhận đặt vé hay không. Điều này giúp tránh trường hợp Agent tự hiểu một yêu cầu đặt vé là đã được người dùng xác nhận.

Ngoài ra, Completion Harness được kiểm tra dựa trên kết quả thực tế của tool, ví dụ BOOKING_SUCCESS, thay vì chỉ dựa vào câu trả lời cuối cùng của LLM. Cách tiếp cận này giúp tăng độ tin cậy của hệ thống.

Ưu điểm

- Kết hợp được tính có cấu trúc của Plan-then-Execute và tính linh hoạt của ReAct.
- Có thể phản ứng với kết quả thực tế của các tool trong quá trình thực thi.
- Permission và Completion được kiểm soát ở tầng chương trình.
- Phù hợp với các tác vụ có nhiều bước như tìm chuyến bay, kiểm tra chuyến bay và đặt vé.

Nhược điểm

- Kiến trúc phức tạp hơn so với ReAct hoặc Plan-then-Execute đơn thuần.
- Có thêm bước tạo plan trước khi thực thi nên có thể tăng thời gian xử lý.
- Có nhiều thành phần cần phối hợp nên việc debug và bảo trì phức tạp hơn.

5. Kết quả thực nghiệm

Để đánh giá ba thiết kế Agent, nhóm thực hiện các kịch bản chính gồm:

- Tìm kiếm chuyến bay.
- Đặt vé khi người dùng chưa xác nhận.
- Đặt vé khi người dùng đã xác nhận.

Kết quả kiểm thử được thực hiện bằng file *tests/evaluate_agents.py*.

5.1. Kết quả ReAct

ReAct Agent vượt qua 4/4 kiểm tra được thiết kế cho Agent:

Kiểm tra	Kết quả
Search flights	PASS
Permission	PASS
Completion	PASS
Handoff	PASS

Kết quả tổng cộng: **4/4 checks passed.**

ReAct cho thấy khả năng xử lý yêu cầu thông qua mô hình reasoning và tool calling. Các Harness cũng có thể được sử dụng để kiểm tra permission, completion và handoff.

5.2. Kết quả Plan-then-Execute

Plan-then-Execute Agent vượt qua 3/3 nhóm kiểm tra:

Kiểm tra	Kết quả
Search flights	PASS
Booking without confirmation	PASS
Booking with confirmation	PASS

Trong trường hợp người dùng chưa xác nhận, Permission Harness từ chối hành động book_flight và hệ thống chuyển sang Handoff. Khi người dùng xác nhận, hệ thống cho phép thực hiện đặt vé và nhận kết quả BOOKING_SUCCESS.

Kết quả tổng cộng: **3/3 checks passed.**

5.3. Kết quả Hybrid

Hybrid Agent cũng vượt qua 3/3 nhóm kiểm tra:

Kiểm tra	Kết quả
Search flights	PASS
Booking without confirmation	PASS
Booking with confirmation	PASS

Đặc biệt, Hybrid Agent kiểm tra Completion dựa trên kết quả thực tế của các tool thay vì chỉ dựa vào câu trả lời cuối cùng của LLM.

Kết quả tổng cộng: **3/3 checks passed.**

5.4. Tổng hợp kết quả

Agent	Số kiểm tra đạt	Kết quả
ReAct	4/4	PASS
Plan-then-Execute	3/3	PASS
Hybrid	3/3	PASS
Tổng cộng	10/10	PASS

```
(.venv) PS C:\Users\ADMIN\Desktop\SE373\BTVM#3 Agent Flight> python tests/evaluate_agents.py
=====
AGENT EVALUATION
=====

Agents:
1. ReAct
2. Plan-then-Execute
3. Hybrid

Scenarios:
A. Search flights
B. Booking without confirmation
C. Booking with confirmation

-----
Agent | Scenario | Result
-----
[1] Evaluating ReAct...
ReAct | Search flights | PASS
ReAct | Booking without confirmation | PASS
ReAct | Booking completion check | PASS
ReAct | Handoff | PASS

[2] Evaluating Plan-then-Execute...
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use
of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
Plan-then-Execute | Search flights | PASS
Plan-then-Execute | Booking without confirmation | PASS
Plan-then-Execute | Booking with confirmation | PASS

[3] Evaluating Hybrid...
Hybrid | Search flights | PASS
Hybrid | Booking without confirmation | PASS
Hybrid | Booking with confirmation | PASS

=====
SUMMARY
=====
ReAct : 4/4 checks passed
Plan-then-Execute : 3/3 checks passed
Hybrid : 3/3 checks passed

-----
Overall: 10/10 checks passed
```

Hình 9. Kết quả tổng hợp các bài test của từng Agent

Như vậy, tất cả các kiểm tra được thực hiện trong bài đều đạt yêu cầu, với tổng cộng **10/10 checks passed**.

Lưu ý rằng số lượng kiểm tra giữa các Agent không hoàn toàn giống nhau, do đó không nên kết luận ReAct tốt hơn chỉ vì có kết quả 4/4 trong khi hai thiết kế còn lại là 3/3. Kết quả trên chủ yếu xác nhận rằng các chức năng và luồng kiểm soát chính đã hoạt động đúng.

6. Đánh giá ba thiết kế

Ba thiết kế có những ưu điểm và hạn chế khác nhau.

Tiêu chí	ReAct	Plan-then-Execute	Hybrid
Tính linh hoạt	Cao	Trung bình	Cao
Luồng thực thi	Khó dự đoán hơn	Rõ ràng	Tương đối rõ ràng
Khả năng phản ứng với tool	Tốt	Hạn chế hơn	Tốt

Kiểm soát Permission	Có thể kiểm tra bằng Harness	Dễ kiểm soát	Dễ kiểm soát
Kiểm tra Completion	Có	Có	Có
Độ phức tạp	Thấp	Trung bình	Cao
Khả năng audit/debug	Trung bình	Tốt	Tốt
Chi phí xử lý	Tương đối thấp	Có thêm bước lập plan	Có thêm bước lập plan và ReAct

ReAct

ReAct có ưu điểm lớn về tính linh hoạt. Agent có thể vừa reasoning vừa lựa chọn tool dựa trên kết quả nhận được. Điều này phù hợp với các tác vụ mà thông tin chưa được biết đầy đủ từ đầu.

Tuy nhiên, luồng thực thi phụ thuộc nhiều vào reasoning của LLM nên khó dự đoán hơn. Nếu chỉ sử dụng prompt để yêu cầu Agent tuân thủ một số quy tắc thì việc đảm bảo các ràng buộc sẽ không mạnh bằng việc kiểm tra trực tiếp bằng code.

Plan-then-Execute

Plan-then-Execute có luồng xử lý rõ ràng hơn. Việc tách quá trình lập kế hoạch và thực thi giúp chương trình dễ kiểm soát, debug và kiểm tra các điều kiện như Permission và Completion.

Nhược điểm là kế hoạch được tạo trước có thể không còn phù hợp nếu thông tin thay đổi trong quá trình thực thi. Ngoài ra, hệ thống cần thêm một bước gọi LLM để tạo plan.

Hybrid

Hybrid kết hợp ưu điểm của hai phương pháp. LLM chịu trách nhiệm lập kế hoạch ở mức cao, trong khi ReAct đảm nhiệm việc tương tác linh hoạt với các tool.

Thiết kế này cũng cho phép đưa các kiểm soát quan trọng xuống tầng chương trình. Trong quá trình triển khai, việc xác nhận đặt vé được kiểm tra bằng code và Completion được xác định từ kết quả thực tế của tool. Điều này giúp giảm sự phụ thuộc vào việc LLM tự suy luận các điều kiện an toàn.

Vì vậy, xét trên yêu cầu của bài tập, **Hybrid là thiết kế cân bằng nhất giữa tính linh hoạt và khả năng kiểm soát**, mặc dù kiến trúc phức tạp hơn hai phương pháp còn lại.

7. Kết luận

Bài tập đã xây dựng thành công một hệ thống Agent đặt vé máy bay sử dụng LangChain với ba thiết kế ReAct, Plan-then-Execute và Hybrid.

Hệ thống đã triển khai đầy đủ bốn cơ chế Harness gồm Data Constraints, Completion Criteria, Permission Check và Handoff. Các cơ chế này giúp hệ thống kiểm soát dữ liệu đầu vào, xác định tác vụ đã thực sự hoàn thành, ngăn các hành động nhạy cảm khi chưa có xác nhận và chuyển tác vụ sang human agent khi Agent không thể tiếp tục.

Ba thiết kế Agent đều được triển khai và kiểm thử. Kết quả thực nghiệm cho thấy toàn bộ các kiểm tra đã thực hiện đều đạt, với tổng cộng **10/10 checks passed**.

Qua quá trình triển khai, có thể thấy ReAct phù hợp khi cần tính linh hoạt, Plan-then-Execute phù hợp khi cần luồng xử lý rõ ràng và dễ kiểm soát, trong khi Hybrid kết hợp được hai ưu điểm này. Đối với bài toán đặt vé máy bay, Hybrid là lựa chọn phù hợp khi cần cân bằng giữa khả năng reasoning linh hoạt của LLM và các cơ chế kiểm soát được thực hiện bằng code.

Một bài học quan trọng là các điều kiện quan trọng như quyền đặt vé và trạng thái hoàn thành không nên chỉ phụ thuộc vào suy luận hoặc câu trả lời của LLM. Các điều kiện này nên được kiểm tra ở tầng chương trình để hệ thống có tính kiểm soát và đáng tin cậy hơn.